D D M M Y Y Y Y

1) Explain the term algorithm with its Criteria Give one example

An Algorithm is a step by step procedure or set of instructions designed to solve a specific problem are perform a task It takes Input processes it through a sequence of steps and produces an output

Criteria of a Good algorithm

A Good algorithm must satisfy the following.

Input - It should accept Zero or more Inputs

output - It must produce at least one output

Definiteness - each steps should be clear precise and unambiguous

finiteness :- each step should be simple and feasible to execute in a resonable time

example of an algorithm.

problem :- find the sum of two numbers.

algorithm → start

Input two numbers A and B

Compute sum = A + B

display Sum.

Stop.

This algorithm takes two Inputs perform addition. and outputs the result

2) Explain each step in fundamentals of algorithm problem solving.

when solving a problem using algorithm we follow a structural approach each step ensures the solution is correct efficient and easy to Implement

Scanned with CamS

* understanding the problem.
clearly identify what needs to be solved.
determine.
Inputs (what data is given)
outputs (what result is expected)
Constraints (limitations like time size etc.
example;- find the largest number in a list
* problem analysis
Breaks the problem into smaller parts
Identify patterns or relationships between inputs and outputs
Decide the type of problem (searching sorting)
* Designing the algorithm
Develop a step by step solution.
choose an approach.
Brute force (simple method)
Divide and Conquer
Greedy method.
Represent the solution using
pseudocode, flowchart.
* Corectance vertification.
check whether the algorithm gives the correct output for al
decide cases
Test with Normal inputs, edges Cases eg:- empty list, large values
* effiency analysis
evaluate performance interms of:-
Time Complexity (how fast it runs)
Space complexity (memory used)
example:- Comparing $O(n)$ vs $O(n^2)$

Scanned with CamS

D D M M Y Y Y Y

* Implementation
Converts the algorithm into a programming language
C, java, python.
follow proper syntax and structure
* Documentation.
write clear explanation about
How to algorithm works
Input/output format

3) Define algorithm explain the steps involved in algorithm design and analysis process with neat diagram.
An algorithm is a finite sequence of well defined step by step instructions used to solve a specific problem or perform a computations It takes Input processes it logical -y and produces output within a finite time.
Neat diagram

Problem solving
↓
Algorithm design.
↓
Correctness check
↓
Complexity analysis
↓
Implementation.
↓
Testing & debugging
↓
Optimisation.

Scanned with Cams

4) Dissus the three asympototic notations with an example

Asymptotic notation are used to analyse the performance time/space complexity of an algorithm as the input size size grows large the three main notations are:-

* Big O Notations - upper Bound.

Represent the worst case complexity of an algorithm

It Gives the maximum time an algorithm can take

def:- An algorithm is O(f(n)) if its running time does not grow faster than f(n)

ex:- Consider, for i = 1 tan:

print (i)

This runs n time → complexity O(n)

even if Input varies it wont exceed more than O(n)

* Omega notation (Ω) → lower bound.

Represent the best case complexity.

It Gives the minimum time required.

def:- An algorithm is Ω(f(n)) if its running time is at least f(n)

ex:- Searching in a Sorted list

if element is found at the first position → Ω(1)

* Theta Notation (Θ) → Tight bound.

Represent Average case/exact bound

It Gives a tight bound where upper bound and lower bounds are the same.

def:- the Algorithm is Θ(f(n)) if it is both.

O(f(n)) & Ω(f(n))

ex:- for i in 1 for n:

print(f)

Always runs n time

so Complexity = O(n)

Scanned with CamS

D D M M Y Y Y Y

5) Explain the Order of Growth with an example.
order the following to their order Growth from the lowest to highest
N-2, logN, N3, 2 powerN, N cube, N Squre, NlogN;
The Order of Growth describes how an algorithm is running time Increases on the Input, size N becomes very large. It helps us compare algorithm based on efficiency
→ It Ignores constants and lower order terms
→ focuses only on the dominant term.
ex:- if time = $3N^2 + 5N + 2$ the Order of Growth is N^2
Consider two algorithm.
Alogrithm A → N type → O(N)
Algorithm B → N^2 steps → $O(N^2)$
for small Inputs both may seems Similar
for large Inputs n^2 grows much faster → algorithm A is more efficient
Ordering the Given functions lowest → highest Growth.
N^{-2}, logN, N^3, 2^N, N, N^3, N^2, NlogN
Correct Order
$N^{-2} < \log N < N^2 < N \log N < N^2 \leq N^3 < 2^N$
N^{-2} → decreases as N Increases
logN → Very slow Growth.
N → linear Growth
NlogN → slightly faster than linear
N^2 → polynomial growth
N^3 → faster polynomial Growth.
2^N → exponential Growth.
logrithmic < linear < polynomial < exponential.

Scanned with CamS

6) With an example discuss mathematical Analysis of an Recursive algorithm

Mathematical analysis of a recursive algorithm means determining its time complexity by expressing its running time as a recursive relation and then solving it

A recursive algorithm calls itself repeatedly so its time depends on.

workdone in each call

Number of recursive calls.

steps in analysis

* Identify the Basic operation.

find the main operative that contributes most to reunning time.

ex:- Comparison addition etc.

* form a recursive relation

express total time $t(n)$ in terms of smaller Inputs

General form $T(n) = at(n/b) + f(n)$

a = number of recursive calls

n/b = size of each subproblem.

$f(n)$ = extra work done.

define Base case

Specify time for smallest input

ex:- $T(1) = C$.

* Solve the Recurrence common method.

substitution method.

Recursion tree

Master theorem

ex:- factorial algorithm

```
fact(n)
  if n==0
```

Scanned with CamS

else:
return * fact (n-1)

step 1 :- Basic operation.
multiplication → executed once per call

step 2 :- Recurence relation
$T(n) = T(n-1) + 1$

step 3 :- Base case
$T(0) = 1$

step 4 : solve Recurrence
expanding
$T(n) = T(n-1) + 1 = T(n-2) + 2 = T(n-3) + 3$

continuing
$T(n) = T(0) + n = 1 + n$

final result
$T(n) = O(n)$ → so the factorial algorithm has linear time complexity.

7) Explain Brute force method write an algorithm for sorting an array using brute force method Analyse it for its complexity.

The brute force method is the most straight forward approach to solving a problem.

It Involves typing all possible solutions and selecting the correct one.

* It does not use any optimisation techniques
* It is easy to understand and Implement
* Usually less efficient for large Inputs

ex:- checking all pairs in an array to find duplicates

Brute force sorting (ex:- Selection Sort)

Scanned with Cams

One common brute force sorting technique in selection sort where we repeatedly find the smallest element and place it in the correct portion.

Algorithm selection sort

```
Selection sort (A, A)
 for i=0 to n-2 do
  min = i
 for J = i+1 to n-1 do
  if A[J] < A [min] then
  min = J
  end for
  swap A[i] and A[min]
  end for
  return A
```

Working explanation:

Step 1: Assume the first element is the smallest
Step 2: Compare it with all other elements
Step 3: find the minimum element
Step 4: Swap it with the first position.
Step 5 : Repeat for remaining elements

* Time Complexity.

outer loop runs → n-1 times
Inner loop runs → n-1-1 times
Total Comparisons.

$(n-1) + (n-2) + \dots + 1 = \frac{n(n-2)}{2}$

∴

Best Case : $O(n^2)$
Average case : $O(n^2)$
Worst Case : $O(n^2)$

Scanned with CamS

Space Complexity.
uses only a few extra variables. O(1) constant space.

8) prove if $t(n) = O(g(n))$ and $t_2(n) = O(g_2(n))$ then
$t(n) + t_2(n) = O(\max(g_1(n), g_2(n))$

Given,
$t_1(n) = O(g_1(n)) = \exists(1, n_1$ such that $t_1(n) \leq c_1 g_1(n) \forall$
$t_2(n) = O(g_2(n)) = \exists(2, n_2$ such that $t_2(n) \leq c_2 g_2(n) \forall$
Add the two inequalities
for $n \geq \max(n_1, n_2)$
$t_1(n) + t_2(n) \leq c_1 g_1(n) + c_2 g_2(n)$
else the property of maximum.
we know
$g_1(n) \geq c_1 \max(g_1(n), g_2(n))$
$g_2(n) \geq c_2 \max(g_1(n), g_2(n))$
so,
$c_1 g_1(n) \leq c_1 \max(g_1(n)\ g_2(n))$
$c_2 g_2(n) \leq c_2 \max(g_1(n), g_2(n))$
Combine.
$t_1(n) + t_2(n) \leq (c_1 + c_2) \max(g_1(n), g_2(n))$
Conclusion:
let $c = c_1 + c_2$, then
$t_1(n) + t_2(n) \leq c \max(g_1(n), g_2(n))$
therefore.
$t_1(n) + t_2(n) = O(\max(g_1(n)\ g_2(n))$
final Result

$$\boxed{t_1(n) + t_2(n) = O(\max(g_1(n), g_2(n))}$$

Scanned with CamS

9) Solve the following Recursive relation
$x(n) = x(n-1) + 5$ for $n>1$, $x(1)=0$

we are given the recursiene

$x(n) = x(n-1) + 5$, for $n>1$, $x(1)=0$

step1 :- Expand the Recurrence.

$x(n) = x(n-1)+5$

$x(n-2)+5+5 = x(n-2)+10$

$x(n-3)+15$

Continuing this process.

$x(n) = x(n-k)+5k$

Step 2: Reach the Base case

let $n-k=1 \rightarrow k=n-1$

$x(n) = x(1)+5(n-1)$

step 3: Substitute Base value

Given $x(1)=0$

$x(n) = 0+5(n-1)$

final answer.

$x(n) = 5(n-1)$

Time Complexity :-

The function grow linearly with n, so $\boxed{x(n) = O(n)}$

10) Solve the following reccursine relation

$T(n)=0$ if $n=0$.

$T(n) = 1 + t(n-1)$ otherwise?

we are Given the recurrence.

$$T(n) = \begin{cases} 0 & n=0 \\ 1+T(n-1) & n>0 \end{cases}$$

step 1: expand the Recurrence

$T(n) = 1+T(n-1)$

$1+(1+T(n-2)) = 2+T(n-2)$

$3+T(n-3)$

Scanned with CamS

D D M M Y Y Y Y

Continuing:

$T(n) = k + T(n-k)$

step 2: Reach the Base case.

let $n-k=0 \rightarrow k=n$

$T(n) = n + t(0)$

step 3: substitute Base value

Given $T(0)=0$.

$T(n) = n+0$

final Answer $= T(n) = n$.

Time complexity.

$\boxed{T(n) = O(n)}$

11) Solve the following Recurrence relation

$x(n) = 3x(n-1)$ for $n>1$, $x(1)=4$

we are given the recurrence

$x(n) = 3x(n-1)$ $n>1$, $x(1)=4$

step 1: expand the recurrence

$x(n) = 3x(n-1)$

$3[3x(n-2)]$

$3^1 2x(n-2)$

$3^2 x(n-3)$

continuing.

$x(n) = 3^k x(n-k)$

Step 2: Reach the Base case

let $n-k=1$

$k = n-1$

$x(n) = 3^{n-1} \cdot 4$

$\boxed{x(n) = 4 \cdot 3^{n-1}}$

Scanned with CamS

Time complexity Growth $\boxed{(x(n)=o(3^n))}$
This shows exponential growth since the value triples at each step.

12) Solve the following recurrence relation
$x(n) = x(n(2)-1$ for $n>1$, $x(1)=1$ where $n=2$ power k
we are Given the recurrence
$x(n) = x(n/2)+1$, $n>1$, $x(1)=1$
step 1: expand the recurrence
$x(n) = x(n/2)+1$
$[x(n1/4)+1]+1$
$x(n/4)+2$
$x(n/8)+3$
continuing.
$x(n) = x(n/2^k)+k$
step 2: Reach the Base case
we stop when
$n/2^k=1$
so, $n=2^k \rightarrow k=\log_2 N$.
step 3: substitute Base value.
$x(n) = x(1)+\log_2 N$
Given $x(1)=1$: $x(n)=1+\log N$
final $x(n)=1+\log_2 N$.
Time Complexity $\boxed{x(n)=o(\log N)}$

13) write an algorithm to find uniqueness find out tim
complexity for an algorithm
Uniqueness problem.
determine whether all elements in a Given array are
distinct or not

Scanned with CamS

D D M M Y Y Y Y

Algorithm (Brute force method)

unique (A, n)

for i = 0 to n-2 do

for J = i+1 to n-1 do

if A[I] = A[J] then.

return Not unique.

end if

end for

end for

return All elements are unique.

explanation:

Compare each element with an elements that come after

if any pair is equal → array is not unique

if no duplicates found → array is unique

Time complexity analysis

1. Worst case

No duplicates → all comparision are mode

$(n-1) + (n-2) + \dots + 1 = n(n-1)/2$

Best Case

Duplicate found at first Comparision

$O(1)$

Average Case

Comparision still grow roughly proportional to n^2

$\rightarrow O(n)^2$.

Space complexity.

No extra space used $\rightarrow O(1)$

Alternative efficient approach using Sorting.

1. Sort the array $\rightarrow O(n \log n)$

2. compare adjacent element $\rightarrow O(n)$

total $O(n \log n)$ (better than brute force)

Scanned with CamS

14) write a algorithm bubble sort find out time complexity efficiently best case, worst case & Average case using bubble sort letter word question sort in alphabetical Order

Bubble sort algorithm

It is a Simple Sorting technique that repeatedly compares adjacent elements and swaps there if they are in the wrong order.

algorithm

```
Bubble sort (A, A)
for i=0 to n-2 do
for J=0 to n-2-i do
if A[J] > A[J+1] then.
  Swap. A[J], A[J+1].
  end if
  end for
  end for.
  return A
```

Time Complexity Analysis

Worst Case.

Array is in reverse order

Maximum comparisons and swaps.

$$\text{Comparison} = \frac{n(n-1)}{2}$$

time complexity $= O(n^2)$

Best Case

Array is already sorted.

if optimised (with a flag) no Swaps occurs.

time Complexity $= O(n)$

Scanned with CamS

D D M M Y Y Y Y

Average Case
Random order
time complexity = $O(n^2)$
Space Complexity
no extra memory required = $O(i)$
ex:- Sorting the word "QUESTIONS"
step 1: Convert to characters
Q, U E S T I O N.
Pass 1:
Compare adjacent and Swap: question
QEUSITON
QESUTION
QESUTION
QESITUON
QESTIOUN
QESIIONU
pass 2:
EQSTIONU
EQSITONU
EQSIOTNU
EQSIONTU
Continue passes.
After all passes.
final Sorted order
EINOQSTU

15) Given matrix multiplication on algorithm find out time Complexity of an algorithm
MYETRY(n):
S<...0

Scanned with CamS

```
for i=0 to n do
    s=s=i*1
    returns
```

i) what does the algorithm compute
ii) what is the basic operation
iii) How many times the basic operation executed
iv) what is the efficiency class of this algorithm

```
Algorithm
Matrix multiply (A,B,n)
for i=0 to n-1 do
for J=0 to n-1 do
  c[i][J]=0
for k=0 to n-1 do
C[i][J]=c[i][J]+A[i][k]* B[K][J]
end for
 end for
 return C
```

Time complexity.

Three nested loops each runs n time total operations $= n \times n \times n = n^3$

Time complexity $= O(n^3)$

Given Algorithm Analysis

```
MYSTRY (n):
  S←0
for i=0 to n do
  S=S≠i*i
  return s
```

i) it computes the sum of squares of first n integer

$S = 0^2 + 1^2 + 2^2 + \ldots + n^2$

$$\boxed{S = \sum_{i=0}^{n} i^2}$$

Scanned with CamS

D D M M Y Y Y Y

ii) The basic operations is
[i x i] multiplication (or) (addition s=s+i*1)
iii) loops runs from 0 to n → total (n+1) times
Basic operation executed
[n+1] times
iv) Number of operations grows linearly with n
time complexity = O(n)

16) find the time complexity of an algorithm for which has two parts i) Sorting part require comparision ii) to check consecutive element require n comparision
Given
An algorithm has two parts
1. Sorting part → requires (n-1) comparisions
2. checking consecutive element → require n comparisions
total comparisions.
total = (n-1) + n = 2n-1
time complexity.
when analysing time complexity we ignore constant and lower order terms.
2n - 1 = O(n)
final answer → time complexity = O(n)
explanation:-
* Both parts individually take linear time
* Adding them still results in linear Growth
* Constants (2) & Constants substraction (-1) are Ignored Big-O notation

Scanned with CamS

17) Write an algorithm for Linear Search find out time complexity for an algorithm

Linear Search is the simplest searching techniques where each element is checked one by one until the desired element is found

Algorithm

```
Linear Search (A, n, key)
for i=0 to n-1 do
 if A[J] == key then
  return i
   end if
   end for
   return -1
```

* Start from the first element
* Compare each element with the Given key
* if found → return its position
* if not found after checking all elements → return -1

Time Complexity.

Best case

elements found at first position

1 Comparision = O(1)

Worst Case

element is last or not present

n Comparisions

O(n)

Average Case

element maybe anywhere in the list

Approx n/2 Comparisions → O(n)

space complexity

No extra memory required

O(1)

Scanned with CamS

D D M M Y Y Y Y

18) write algorithm selection sort find out the time complexity efficency best case worst case & Average case using selection sort letter word EXAMPLE Sort in alphabetical order.

selection sort words by repeatedly selecting the smallest element from the unsorted part & playing it act the correct position.

Algorithm

```
Selection sort (A,n)
 for i=0 to n-2 do
 min = i
 for J = i+1 to n-1 do
 if A[J] < A[min] than
 min = J
  end for
 Swap A[i] = A[max]
 end for
 returns A
```

Time Complexity analysis

Number of comparison.

$(n-1)+(n-2)+\ldots+1 = n(n-1)/2$

Best Case : Array already sorted
Still compares all elements

$O(n^2)$

Worst case : Array in reverse order.

$O(n^2)$

Average case : Random Order

$O(n^2)$

Space Complexity)

$O(1) \rightarrow$ (in place sorting)

example : Sorting EXAMPLE

Scanned with CamS

Initial array.
E X A M P L E
Pass 1 (i=0) → Smallest A → A, X, E, M, P, L, E
Pass 2 (i=1) → Smallest E → A, E, X, M, P, L, E
Pass 3 (i=2) → Smallest E → A, E, E, M, P, L, X
Pass 4 (i=3) → Smallest L → A, E, E, L, M, P, X
Pass 5 (i=4) → Smallest M → A, E, E, L, M, P, X
Pass 6 (i=5) → Smallest P → A, E, E, L, M, P, X
final Sorted Order →
A E E L M P X

19) Mention General plane for the f Analysing the efficiency of measure algorithm apply this plane for analysing the recursive algorithm to solve factorial on N!
General plan for analysing the efficiency of an algorithm
The efficiency of an algorithm is analysed using a Systematic approach.
* Identify Input size (n)
Determine the parameter that represent Input size
ex:- Number of elements n.
* Identify the Basic operations.
find the most frequently executed operation
ex:- Comparison multiplication addition
* Count the Number of Basic operations
Determine how many times the Basic operation is executed as a function of n
for recursive algorithm form a recursive relation
* Consider Best worst and average case.
Best Case : Minimum operation
Worst Case : Maximum operation.

Scanned with CamS

Average Case : expected operation

* Derive time Complexity.
express growth using asymptotic notation:
$O(\cdot)$ $\Omega(\cdot)$, $\theta(\cdot)$

* optional, space Complexity.
Analyse memory usage if required.

Apply this plan to recursive factorial algorithm

Algorithm
fact(n)
if n == 0 then
return 1
else
return n * fact(n-1)

step 1 :- Input size.
input is n

step 2 :- Basic operation.
Multiplication → n × fact(n-1)

step 3 :- Count operations (Recurrence Relation)
$T(n) = T(n-1) + 1$ Base case $T(0) = 1$)

step 4 : Solve recurrence
expanding.
$T(n) = T(n-1) + 1 \rightarrow T(n-2) + 2 \rightarrow T(0) + n$
$T(n) = 1 + n$

step 5 :- Time Complexity
$T(n) = O(n)$

step 6 :- Space Complexity.
due to recursion stack $O(n)$

Scanned with CamS

20) Write the brute force string matching algorithm find out time complexity determine the number of comparisons required in searching for the pattern CSE using the text VTUSCSESCOURCE

Brute force string matching algorithm.

this method checks for the pattern at every possible position in the text

Algorithm:-

```
Brute force string Match (T. P)
n = length (T)
m = length (P)
for i = 0 to n-m do
J=0
while J<m & T[i+J]=P[iJ do
 J=J+1
end while.
if J = km then
 return i
 end for
  return 1
```

Time complexity

Bust case: pattern matches immediatly.

O(m)

Worst Case: All alignments checked.

$(n-m+1)\times m \simeq O(nm)$

Average Case

$o(nm)$

Given problem.

Text (T) = VTU SCSESCOURCE

pattern (p) = CSE

Scanned with CamS

D D M M Y Y Y Y

length

n = 15 . m = 3

step by step matching

we compare pattern of each position

i = 0 VTU → mismatch (1 comparision)

i = 1 TVS → mismatch (1 comparision)

i = 2 VSC → mismatch (1 comparision)

i = 3 SCS → mismatch (1 comparision)

i = 4 CSE → match (3 comparisions)

total comparisons:-

1 + 1 + 1 + 1 + 3 = 7

final Answer = Number of comparision : 7

Time Complexity = O(nm)

21) Consider the algorithm

```
SECRET (n,0):
min val < 0
for i=0 to n-1 do
  if [a[i] < min val)
    min val = a[i].
  if (a[i] > max val)
    max val = a[i]
```

i) what does the algorithm compute

ii) what is the basic operations

iii) How many times the basic operations executed

iv) what is the efficiency class of this algorithm

The Given algorithm is essentially

```
Secret (n, 0)
min val ← a[0]
max val ← a[0]
```

Scanned with CamS

```
for i=0 to n-1 do
if (a[i] < min val)
    minval = a[i]
if (a[i] > max val)
    max val = a[i].
```

i) this algorithm find the maximum and minimum elements in the array of size n

ii) the basic operations are the comparisions a[i]<minval a[i]>maxval these are executed inside the loop

iii) Number of times the basic operations is executed for each iteration 2 comparisions are made.
Total interactions = n
so, total Comparision = 2n

iv) the time complexity is based on no of comparision
$T(n) = 2n \rightarrow O(n)$ so, the algorithm belongs to the linear time Complexity class

22) write an algorithm for maximum of n element find the time efficiency of an algorithm

Algorithm to find maximum of n elements

```
MAXIMUM(A,n)
max <- A[0]
for i = 1 to n-1 do
    if A[i] -> max then
        max <- A[i]
    return max
```

Assume the first element is the maximum
Compare it with each of the remaining elements
update max whenever a large elements is found

Scanned with CamS

D D M M Y Y Y Y

Time efficiency analysis

Basic operations

* Comparision : A[i] > max

Number of Comparisions

* loops runs from 1 to n-1
* So, Comparisions = (n-1)

Time Complexity

* Best Case : O(n)

Worst Case : O(n)

Average Case : O(n)

efficiency class

* linear time complexity → O(n)

23) a) Explain and write merge sort algorithm

b) find out time complexity of merge sort using

i) master theorem ii) substitution method.

Master sort → explanation

It is a divide and conquer algorithm it works in three steps

divide : split the array into two halves

conquer : Recursively sort both halves

Combine (Merge) : Merge the two Sorted halves into one sorted array

It repeatedly divides the array we until each Sub array has one element the merges them in sorted array

Algorithm : Merge Sort

MERGE SORT (A, low, high)

if low < high

mid <— (low + high)/2

Scanned with CamS

Merge = Sort (A, low, mid)
Merge = Sort (A, mid+1, high)
Merge = Sort (A, low, mid, high)

Algorithm :- Merge procedure

Merge (A, low, mid, high)
create temporary arrays L & R
copy A[low, mid] into L
copy A[mid+1 ... high] into R
i ← 0, J ← 0, k ← 100
while i < size (L) & J < size (R) do
if L[i] = R[J] then.
A[k] ← L[J]
i ← i+1
else
A[k] ← R[J]
J ← J+1
k ← k+1
copy remaining elements of L (if only)
copy remaining elements of R (if only)

Time Complexity Analysis

Recurrence Relation.

Merge sort divides the problem into 2 Subproblems and merging takes linear time.

$T(n) = 2T(n/2) + n$

i) using master theorem

Compare with standard form.

$T(n) = aT(n/b) + f(n)$.

$a = 2 \quad b = 2 \quad f(n) = n$

Now

$n \log a = n \log^2 = n$

Scanned with CamS

So, $f(n) = \Theta(n \log_b a)$

This is cause 2 of master theorem

Result $T(n) = \Theta(n \log n)$

ii) using Substitution method

Given $T(n) = 2T(n/2) + n$

expand $2[2T(n/4) + n/2] + n$

$4T(n/4) + 2n$

$8T(n/8) + 3n$

again: $8T(n/8) + 3n$.

After k steps $2^k T(n/2^k) + kn$

let $n/2^k = 1 \rightarrow k = \log n$

So,

$T(n) = nT(1) + n\log n$

$T(n) = \Theta(n \log n)$

24) a) Explain and write Binary Search algorithm

b) Find out complexity Quick Sort using.

i) Master theorem ii) Substitution method.

Binary search is an efficient Searching algorithm used on a Sorted array

Working

find the middle element

if Key == middle → element found.

if Key ≤ middle → Search left half

if Key > middle → Search right half

Report until found or range become empty.

it reduces the Search space by half each time.

Scanned with CamS

Algorithm:- Binary Search

```
BINARY-SEARCH (A, low, high, key)
while low ≤ high do
    mid ← (low+high)/2
    if A[mid] = key then
        return mid
    else if key < A[mid] then
        high ← mid-1
    else
        low ← mid-1
return -1
```

Time Complexity of Binary Search

each step reduces size to n/2

Recurrence $T(n) = T(n/2) + 1$.

Solution $T(n) = o(\log n)$

Time Complexity of Quick Sort

Recurrence Relation → $T(n) = T(k) + T(n-k-1) + n$

Best case $T(n) = 2T(n/2) + n$

Worst Case $T(n) = T(n-1) + n$.

i) using master theorem

$T(n) = 2T(n/2) + n$

$a=2, b=2, f(n) = n$

$n \log_b a = n$.

Case 2 applies

$T(n) = \theta(n \log n)$

ii) Using Substitution method.

Best Case

$T(n) = 2T(n/2) + n$

expanding $= 2(2T(n/4) + (n/2) + n$

$4T(n/4) + 2n$

$8T(n/8) + 2n$

Scanned with CamS

After k steps $= 2^k T(n/2^k) + kn$

let $n/2^k = 1 \rightarrow k = \log n$

$T(n) = nT(1) + n\log n$

$T(n) = \Theta(n \log n)$

Worst Case :-

$T(n) = T(n-1) + n$

expand $= T(n-2) + (n-1) + n$

$T(n-2) + (n-1) + n$

continue

$T(1) + 2 + 3 + \ldots + n$

$\Theta(n^2)$

25) a) 20 sort array list using quick sort 65 74, 75, 80, 85, 60, 55, 50, 45

b) sort array list using quick sort P, r, O, g R, a, M, I, N, G

— a) Quick Sort on Numbers

Given array : 65, 70, 75, 80, 85, 60, 65, 55, 50, 45

step 1 :- pivot = 65

left (<65) : 60, 55, 50, 45

right (>65) : 70, 75, 80, 85

Array becomes : [60, 55, 50, 45] 65 [70, 75, 80, 85]

step 2 :- pivot = 60

left : 55, 50, 45

Right

→ [55, 50, 45] 60

step 3 :- pivot = 55

left = 50, 45

[50, 45] 55

step 4 :- pivot = 50

left : 45

Scanned with CamS

D D M M Y Y Y Y

45,50
Sorted left side = 45,50,55,60
Step 5:- pivot = 70.
Right = 75, 80, 85
70 [75, 80, 85]
Continue similarly -> already Sorted
70, 75, 80, 85
final Sorted array: 45, 50, 55, 60, 65, 70, 75, 80, 85

b) Quick sort on characters.
Given array
P r D G R O M m N g.
inter prefing your Input properly.
Sorting is based on Asc II values
upper Case (A-7) come before lowe case (a-7) step (concept)
final sorted Order G M N O P R a g m r

26) a) find out Binary Search time Complexity.
b) Construct heap using top down approach for the list 14, 12, 9, 8, 7, 10, 18
c) Apply strassens algorithm to multiply the Given two matrices

— a) Binary Search time complexity.
Binary Search Works by dividing the array into halves
Recurrence relation
$T(n) = T(n/2) + 1$
Solving.
$T(n) = O(\log n)$

Scanned with CamS

D D M M Y Y Y Y

Best Case = O(1) element found at middle
Worst Case = O(log n)
Average Case = O(log n)

b) Construct heap (Top down Approach)
Given list: 14, 12, 9, 8, 7, 10, 18 [construct max heap using Insertion method]
step by step Insertion
Insert 14 → [14]
Insert 12 → [14, 12]
Insert 9 → [14, 12, 9]
Insert 8 → [14, 12, 9, 8]
Insert 7 → [14, 12, 9, 8, 7]
Insert 10 → [14, 12, 10, 8, 7, 9]
Compare with parent (9), swap
Insert 18 → Compare with 10, swap
→ Compare with 14, swap
[18, 12, 14, 8, 7, 9, 10]
final max heap → 18, 12, 14, 8, 7, 9, 10.

c) Strassen's matrix multiplication
Strassen's algorithm is used to multiply two matrices efficiently
Divide matrices into submatrices
for 2×2 matrices

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

Compute 7 products

Scanned with CamS

D D M M Y Y Y Y

Compute 7 products

$M_1 = (a+d)(e+h)$

$M_3 = a(f-h)$

$M_5 = (a+b)h$

$M_7 = (b-d)(g+h)$

Compute Result matrix

$C_1 = M_1 + M_3 \; M_5 + M_7$

$C_2 = M_2 + M_4$

27) a) write an algorithm for job sequencing using greedy Algorithm

b) Solve the knapsack problem obtain the optimal Solution using array greedy technique Given problem

$M = 400 = 3$

$W_1, W_2, W_3 = 20, 25, 10$

$P_1, P_2, P_3 = 30, 40, 35$

c) what is the Solution for job sequencing problem with deadlines when $n = 7$

$P_1\ P_2\ P_3\ P_4\ P_5\ P_6\ P_7$ 3,5, 20, 18, 1, 6, 30

$d_1\ d_2\ d_3\ d_4\ d_5\ d_6\ d_7 = 1, 3, 4, 3, 2, 6, 2$

– a) job sequencing with deadlines - Greedy algorithm

Idea:- Selecting jobs in decreasing order of profit & assign each job to the latest possible free slot before its deadline

Algorithm:-

JOBSEQUENCING (J, n)

Sort jobs in decreasing order of profit

Create an array slot [1...n] and Initalise to empty

profit ← 0

for i = 1 to n do

Scanned with CamS

D D M M Y Y Y Y

for J = min(n, deadline[i] down to 1 do
  if slot[i] is empty then
    slot[J] ← job[i]
    profit ← profit + profit[i]
    break.
return slot, profit

b) fractional knapsack (Greedy method)

Given:

Capacity M = 40

weight : W = [20, 25, 10]

profit : P = [30, 40, 35]

Step 1 : Compute profit [weight Ratio].

item	weight	profit	Ratio
1	20	30	1.5
2	25	40	1.6
3	10	35	3.5

Step 2 : Sort by Ratio

item 3 → item 2 → item 1

Step 3 :- fill knapsack

Take item 3 fully → weight = 10, profit = 35

Take item 2 fully → weight = 25, profit = 40.

Remaining Capacity = 5

Take 5/20 of item 1

profit = 30 × (5/20) = 7.5

Total profit = 35 + 40 + 7.5 = 82.5

optimal Solution profit = 82.5

Scanned with CamS

c) Job Sequencing problem solution

Given

profit $P = \{3, 5, 20, 18, 1, 6, 30\}$

Deadlines : $d = [1, 3, 4, 3, 2, 6, 2]$

Step 1: Sort by profit (descending)

Job	profit	Deadline
J_7	30	2
J_3	20	4
J_4	18	3
J_6	6	6
J_2	5	3
J_1	3	1
J_5	1	2

Step 2:- Assign slots (max deadline = 6)

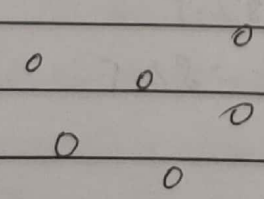
slot	job
1	J_1
2	J_7
3	J_4
4	J_3
5	–
6	J_5

selected jobs = J_7, J_3, J_4, J_6, J

Total profit = $30+20+18+6+3=77$

28) a) write a Topological Sort Algorithm

b) find the topological order of the Given Graph by using source removal method.



Scanned with CamS

Algorithm : Topological Sort (Source removal method)

```
Topological-Sort(G)
    Compute in-degree of all vertices
    Create a queue Q
    enqueue all vertices with in-degree = 0
    while Q is not empty do
        v <= dequeue(Q)
        print v
        for each adjacent vertex u for v do
            reduce in-degree of u by 1
            if in degree of u becomes 0 then
                enqueue(u)
```

Source removal method.

Source vertex is one with in-degree = 0

Remove the source and add it to the ordering.

update in-degree of remaining vertices

Repeat until all vertices are removed.

About the Given Graph.

* The Image you provided shows only vertices but no directed edges (arrows) are visible.
* Topological sorting requires direct edges without edges the Order cannot be uniquely determined
* if no edges exist then Any order of vertices is so valid topological order

Scanned with CamS